

||||

=====

INBOUND LAYER (The Receptionist)

=====

[Client Application]

|

| 1. Submit Heavy Request (e.g., File, Raw Data, Command)

v

[Ingestion API]

|

| 2. Validates payload structure

| 3. Generates unique Tracking ID

| 4. Writes record to State Ledger with status: "QUEUED"

| 5. Pushes Tracking ID + Payload Data into the Message Buffer

v

[Ingestion API] ---> Returns Instant Response ---> [Client Application]

(HTTP Status: 202 Accepted)

(Payload: {"tracking_id": "XYZ"})

=====

BUFFER & STORAGE LAYER (The Vault)

=====

[Message Buffer] <--- Holds raw, unexecuted task references sequentially.

Acts as a structural shock absorber.

[State Ledger] <--- Persistent database. Holds the centralized truth of what
state ("QUEUED", "RUNNING", "DONE", "ERROR") a task is in.

=====

EXECUTION LAYER (The Factory Floor)

=====

[Message Buffer]

|

| 6. Pulls next available task (Controlled Concurrency / Capped Throughput)

v

[Processing Worker]

|

| 7. Updates State Ledger entry to: "RUNNING"

| 8. Executes the heavy computation (Isolated from the Ingestion API)

| 9. Updates State Ledger entry to: "DONE" or "ERROR"

v

[State Ledger]

=====

OUTBOUND LAYER (The Follow-up)

=====

[Client Application]

|

| 10. Check Status (Polls GET /status/XYZ or listens via open persistent socket)

v

[Ingestion API]

|

| 11. Reads current state directly from State Ledger

v

[Ingestion API] ---> Returns Response ---> [Client Application]

(HTTP Status: 200 OK)

(Payload: {"status": "RUNNING"})

+-----+

| Client |

+-----+

|

| 1. Submit Payload

v

+-----+

| API Gateway |

+---+-----+---

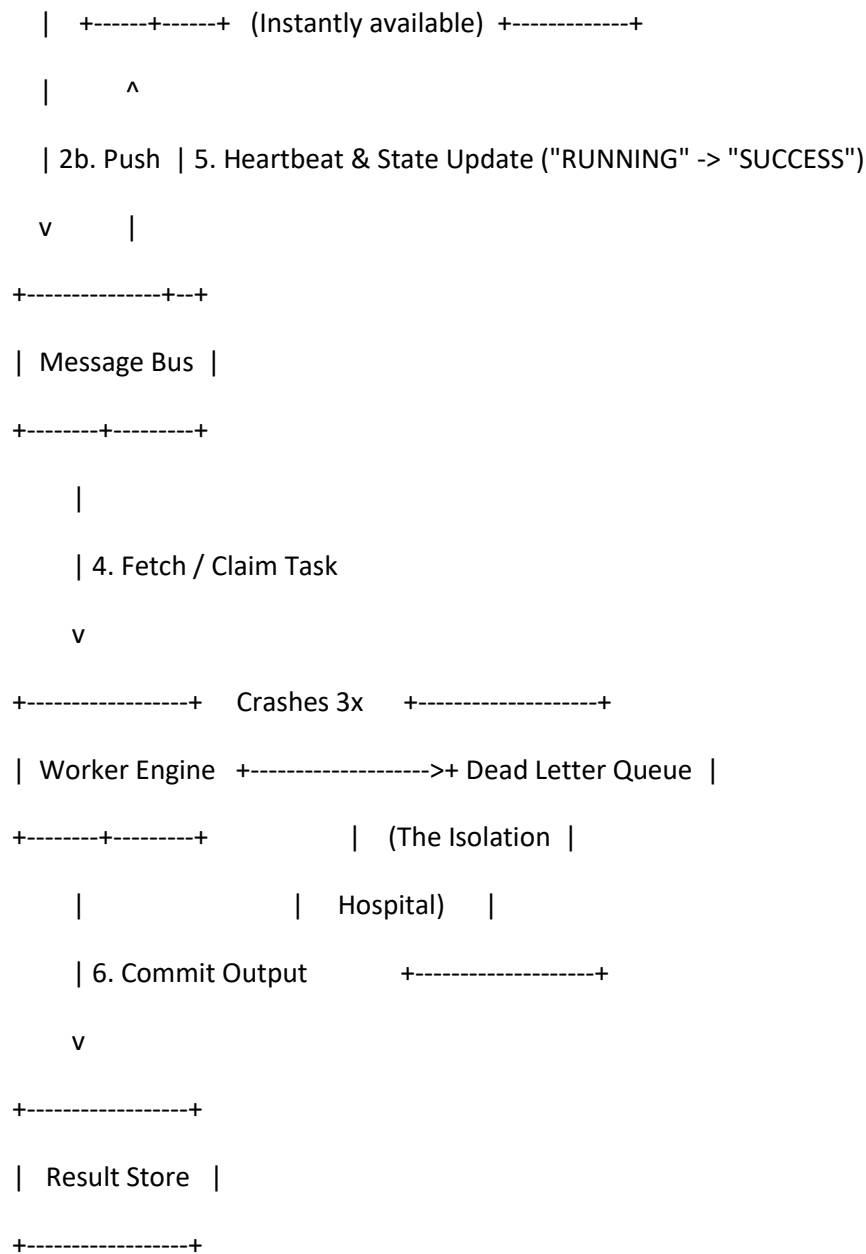
| |

| 2a. Write State: "QUEUED"

| v

| +-----+ 3. Read Status +-----+

| | State Store | <-----+ Client |



Project Summary & Core Architecture:

The Ingestion Gateway (API): Receives the payload, validates it, generates a UUID, writes a PENDING state to the Database, pushes the UUID to the Queue, and immediately returns a 202 Accepted to the client.

The Buffer (Message Queue): Holds the backlog of job references.

It acts as the shock absorber for traffic spikes.

The Execution Engine (Workers): Independent processes that pull tasks from the queue, update the database to PROCESSING, execute the raw computation, and update the database to COMPLETED or FAILED.

The Source of Truth (State Database): A persistent ledger independent of the queue.

If a client queries GET /status/{uuid}, the API reads it directly from here, entirely bypassing the queue and workers.

Worst-Case Analysis:

The Poison Pill (Bad Data):

What breaks: A user submits a corrupted file.

The worker tries to process it, throws an unhandled exception, and crashes.

The queue hands the same job to the next worker, which also crashes.

Your whole worker pool dies.

The Requirement: Implement a retry limit (e.g., 3 attempts).

If it fails 3 times, route it to a Dead-Letter Queue (DLQ) and mark it

FAILED in the database so engineers can debug it later.

The Zombie Worker (Network/Hardware Hang):

What breaks: A worker picks up a job,

updates the state to PROCESSING, but the server loses network connection.

The worker never reports back. The job is stuck in PROCESSING forever.

The Requirement: Implement Visibility Timeouts or Heartbeats.

If a worker hasn't reported back in X seconds, the system assumes it's dead,

unlocks the job, and re-queues it.

The Double-Charge (Lack of Idempotency):

What breaks: A worker finishes a job (e.g., transferring money),
but crashes right before telling the queue it finished.

The queue gives the job to another worker. The money is transferred twice.

The Requirement: The worker must check the State Database before executing.

If the database says COMPLETED, the worker immediately drops the task.

!!!!